

# Stack Memory with Push and Pop Operations

Student Name and Surname | Student ID

Burak Yiğit Hacıoğlu 22040102014

Ezgi Öz dabakoğlu 22040102032

Alperen Korkmaz 22040102019

GitHub:

<https://github.com/burakyigithacioglu>

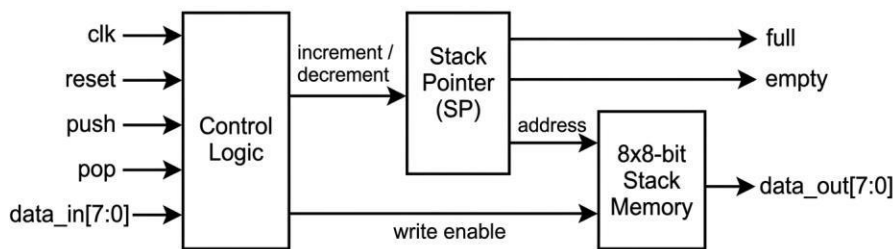
<https://github.com/ezgiozdabakoglu>

<https://github.com/alperenkorkmaz09>

## 1. Introduction

This project implements a stack memory using Verilog. A stack is a memory structure that follows the LIFO principle, which means Last In First Out. In this design, data can be inserted into the stack using the push operation and removed using the pop operation.

## 2. Digital Design



The system consists of three main parts: control logic, stack pointer, and memory array. The control logic manages push and pop operations. The stack pointer keeps track of the current top position of the stack. The memory array stores 8-bit data values. The full and empty signals indicate the status of the stack.

Figure 1: Block Diagram of Stack Memory System

### 3. Verilog Implementation

#### stack\_memory.v

```
module stack_memory (  
    input clk,  
    input reset,  
    input push,  
    input pop,  
    input [7:0] data_in,  
    output reg [7:0] data_out,  
    output full,  
    output empty  
);  
  
    reg [7:0] memory [0:7];  
    reg [3:0] sp;  
  
    assign empty = (sp == 0);  
    assign full = (sp == 8);  
  
    always @(posedge clk or posedge reset) begin  
        if (reset) begin  
            sp <= 0;  
            data_out <= 8'b00000000;  
        end  
        else begin  
            if (push CC !full CC !pop) begin  
                memory[sp] <= data_in;  
                sp <= sp + 1;  
            end  
            else if (pop CC !empty CC !push) begin  
                sp <= sp - 1;  
                data_out <= memory[sp - 1];  
            end  
        end  
    end
```

```
        end
    end
end
```

```
Endmodule
```

#### 4. Verilog Testbench

The Verilog module includes clk, reset, push, pop, and data\_in as input signals. The outputs are data\_out, full, and empty. The memory array has 8 locations, and each location stores 8-bit data.

#### stack\_memory.v

```
`timescale 1ns/1ps
```

```
module stack_memory_tb;
```

```
    reg clk;
    reg reset;
    reg push;
    reg pop;
    reg [7:0] data_in;
```

```
    wire [7:0] data_out;
    wire full;
    wire empty;
```

```
    stack_memory uut (
        .clk(clk),
        .reset(reset),
        .push(push),
        .pop(pop),
        .data_in(data_in),
        .data_out(data_out),
```

```
.full(full),  
.empty(empty)  
);
```

```
always #5 clk = ~clk;
```

```
initial begin  
    $dumpfile("stack_memory.vcd");  
    $dumpvars(0, stack_memory_tb);
```

```
    clk = 0;  
    reset = 1;  
    push = 0;  
    pop = 0;  
    data_in = 8'd0;
```

```
    #10;  
    reset = 0;
```

```
    #10; data_in = 8'd10; push = 1;  
    #10; push = 0;
```

```
    #10; data_in = 8'd20; push = 1;  
    #10; push = 0;
```

```
    #10; data_in = 8'd30; push = 1;  
    #10; push = 0;
```

```
    #10; pop = 1;  
    #10; pop = 0;
```

```
    #10; pop = 1;  
    #10; pop = 0;
```

```

#10; pop = 1;

#10; pop = 0;


#10; pop = 1;

#10; pop = 0;


#10; data_in = 8'd1; push = 1;

#10; data_in = 8'd2;

#10; data_in = 8'd3;

#10; data_in = 8'd4;

#10; data_in = 8'd5;

#10; data_in = 8'd6;

#10; data_in = 8'd7;

#10; data_in = 8'd8;

#10; push = 0;


#10; data_in = 8'd99; push = 1;

#10; push = 0;


#20;

$finish;

end

```

Endmodule

## 5. Simulation Results

The waveform result shows that the stack works correctly. The values 10, 20, and 30 are pushed into the stack. During pop operations, the output values are 30, 20, and 10. Therefore, the waveform proves the Last In First Out behavior.

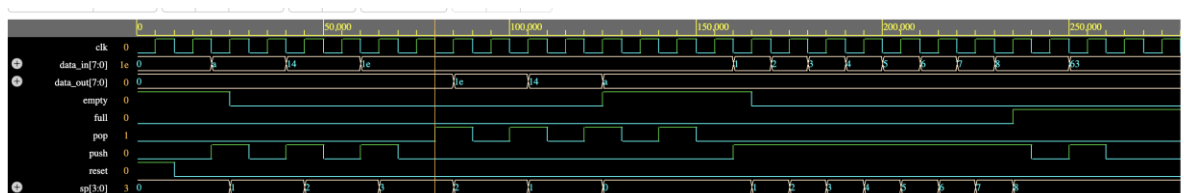


Figure 2: Waveform showing push, pop, empty, and full conditions of the stack memory.

## 6. Conclusion

The stack memory system has been successfully designed, implemented, and tested using Verilog. The simulation results confirm that the system works correctly according to the LIFO principle.